DATA SCIENCE REPORT

MANUFACTURING DEFECT INTELLIGENCE: DEFECT SEVERITY CLASSIFICATION & QUALITY ANALYTICS

Defect Severity, Quality Analytics & Predictive Modeling



TOOLS USED

Python, Pandas, Numpy, Matplotlib, Plotly & ML libraries

PRESENTED BY

Tejas Jadhav

GITHUB

TABLE OF CONTENTS

Overview & Business Objective	3
Data Understanding & EDA	8
Data Preparation & Feature Engineering	23
Model Development & Experimentation	27
Model Evaluation & Comparison	32
Explainability & Final Model	41
Business Insights & Recommendations	47
Results & Business Insights	52
Limitations & Future Scope	57
Conclusion	61

Overview & Business Objective

1.1 PROJECT OVERVIEW

Manufacturing quality assurance plays a critical role in maintaining product reliability, identifying defects, and supporting effective inspection and repair decisions. Defects vary in type, location, and severity, making accurate severity classification an important aspect of quality management.

Understanding these defect characteristics can help manufacturers assess quality risks, prioritize inspections, and improve defect management processes.

The Manufacturing Defect Intelligence project applies machine learning to manufacturing defect records to analyze defect patterns and classify severity into Critical, Minor, and Moderate categories.

The project combines:

- Data understanding and cleaning
- Exploratory Data Analysis
- Behavioral feature engineering
- Class imbalance handling
- Machine learning classification
- Model evaluation and comparison
- Prediction-error analysis
- Model explainability
- Business-oriented insights

The objective is not only to investigate the feasibility of predicting defect severity, but also to evaluate classification reliability, understand the factors influencing model predictions, and identify opportunities for improving manufacturing quality assurance.

1.2 PROBLEM STATEMENT

Manufacturing organizations must evaluate defects and determine their severity to support inspection prioritization, repair planning, and quality management.

Severity assessment may depend on multiple factors, including defect characteristics, defect location, inspection procedures, and manufacturing conditions. When these relationships are complex or insufficiently documented severity classification can become challenging.

The central problem addressed in this project is:

Can manufacturing defect characteristics and inspection information be used to reliably classify defects into Critical, Minor, and Moderate severity levels, while identifying the limitations and factors influencing model predictions?

The project formulates this task as a supervised multiclass classification problem, with ***severity*** as the target variable.

Particular attention is given to the identification of Critical defects, where incorrect predictions may have greater operational consequences than ordinary classification errors.

1.3 PROJECT OBJECTIVES

The major objectives of the project are:

- Examine the structure, quality, and characteristics of the manufacturing defects dataset.
- Investigate defect distributions, severity patterns, inspection methods, and repair costs through exploratory data analysis.
- Identify suitable predictive variables while excluding identifiers and potentially leakage-prone features.
- Engineer temporal features from recorded defect dates.
- Develop a reproducible preprocessing workflow for numerical, categorical, and binary variables.
- Train and compare multiple machine learning classification algorithms.
- Optimize selected models through hyperparameter tuning and stratified cross-validation.
- Investigate whether voting and stacking ensembles improve classification performance.
- Evaluate model generalization using a separate held-out test dataset.
- Examine Critical defect recall, false alerts, and missed severe defects.
- Explain model behavior using coefficients, feature importance, permutation analysis, and SHAP.
- Translate the findings into practical quality assurance considerations and identify opportunities for future improvement.

1.4 PROJECT SCOPE

The project follows a structured workflow covering data analysis, behavioral feature engineering, machine learning, model evaluation, and explainability to understand purchase patterns, identify key conversion signals, evaluate predictive performance, and derive actionable e-commerce insights.

01

Data Analysis

- Dataset inspection
- Data quality assessment
- Defect distribution analysis
- Severity and repair cost analysis

02

Predictive Modeling

- Feature preparation
- Temporal feature engineering
- Multiclass classification
- Hyperparameter tuning and ensemble learning

03

Explainability

- Model performance comparison
- Critical defect detection analysis
- Error analysis & SHAP Analysis

04

Business Interpretation

- Defect severity insights
- Inspection and quality risk assessment
- Classification reliability analysis
- Recommendations for quality improvement

Data Understanding & Exploratory Data Analysis

2.1 DATA OVERVIEW

The project uses the Manufacturing Defects Dataset, a synthetic dataset containing records of manufacturing defects and their associated characteristics.

Data Source: Kaggle

Dataset Size

1,000
Records

8
Original Features

Purchase Diversity

3
Severity Levels

3
Inspection Methods

3
Defect Types



DATA OVERVIEW

Below is a detailed description of the feature set:

Dataset Features	Type	Feature Description
defect_id	Categorical (Identifier)	Unique identifier assigned to a defect record
product_id	Categorical (Identifier)	Identifier of the associated product
defect_type	Categorical (Nominal)	Classification of the manufacturing defect
defect_date	Date	Date associated with the recorded defect
defect_location	Categorical (Nominal)	Location of the defect within the product
severity	Categorical (Ordinal)	Severity classification of the defect (Target Variable)
inspection_method	Categorical (Nominal)	Method used to inspect or identify the defect
repair_cost	Numerical (Continuous)	Recorded cost associated with defect repair

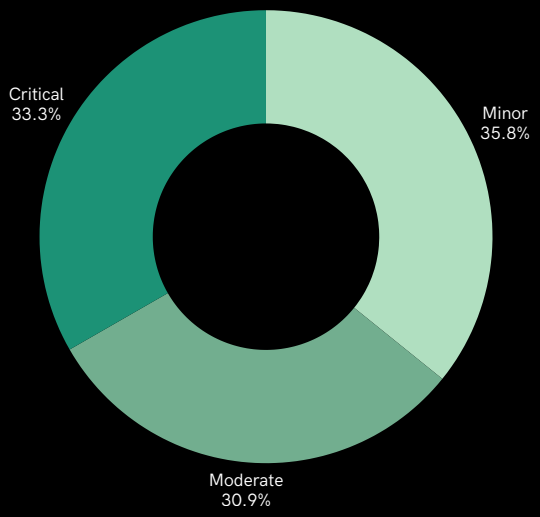
2.2 TARGET VARIABLE

The target variable selected for classification is:

severity

It represents the recorded severity classification of each manufacturing defect.

The dataset contains three severity categories:



The severity classes are relatively balanced, with only modest differences in representation.

This is useful for multiclass modeling because no single class overwhelmingly dominates the dataset. Nevertheless, balanced class frequencies do not guarantee that the classes are distinguishable using the available features.

2.3 DATA QUALITY & PREPARATION OVERVIEW

Before modeling, the dataset was examined for:

- Data types
- Missing values
- Duplicate observations
- Categorical variables
- Numerical variables
- Target variable distribution
- Potential outliers
- Feature relationships
- Class imbalance

No missing values were identified in the dataset.

Several variables required particular consideration before model development.

defect_id was excluded because it serves as a unique record identifier rather than a meaningful description of defect severity.

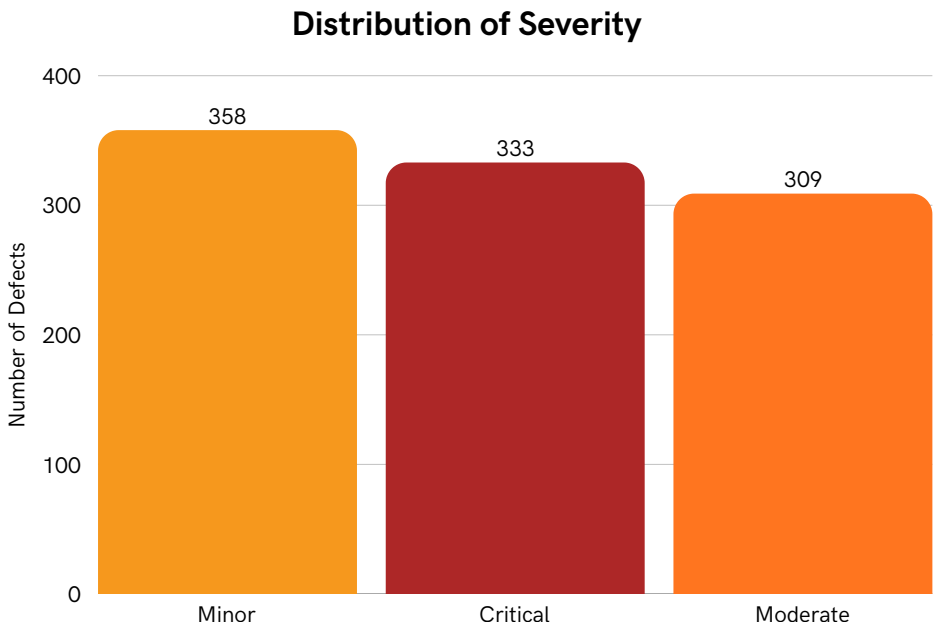
product_id was excluded because of its high cardinality and the absence of additional product-level information supporting its use.

repair_cost was excluded from predictive modeling because repair costs may be determined after defect severity assessment. Including such information could create target leakage in a system intended to classify defects before repair decisions are made.

2.4 Exploratory Data Analysis

Defect severity is relatively evenly distributed, with Minor defects accounting for the largest share

~25%
New Visitor Conversion



Key observations

- Minor defects account for 35.8% of observations, followed by Critical defects at 33.3% and Moderate defects at 30.9%.
- The three severity classes have relatively similar frequencies, indicating no substantial class imbalance.

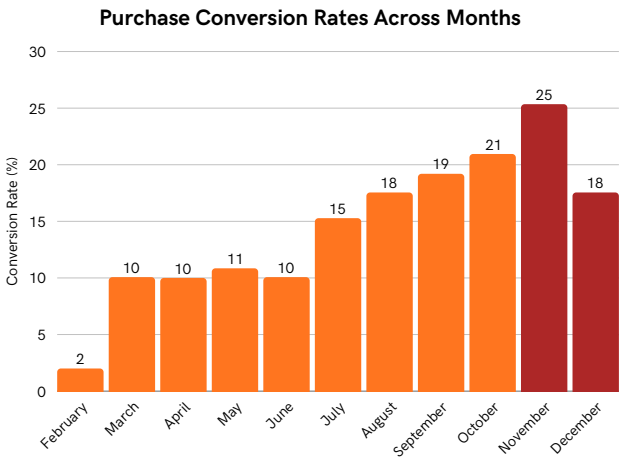
Business Insights

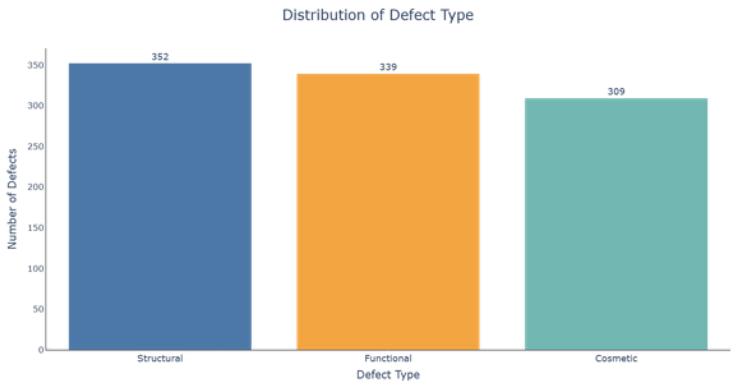
- Critical defects represent approximately one-third of recorded defects, making their identification an important quality assurance priority.
- The balanced distribution supports comparisons across severity categories, although accurate classification still depends on the availability of informative predictors.

Distribution of Defect Type

25.35%

November Conversion Rate





Key observations

- Structural defects account for 352 observations (35.2%), followed by Functional defects at 339 (33.9%) and Cosmetic defects at 309 (30.9%).
- All three defect categories have comparable representation, with Structural defects occurring slightly more frequently.

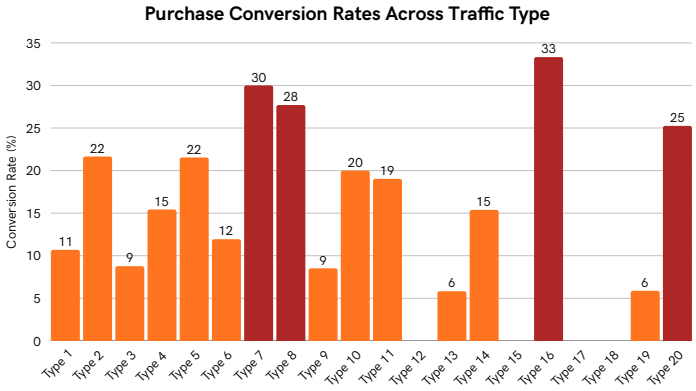
Business Insights

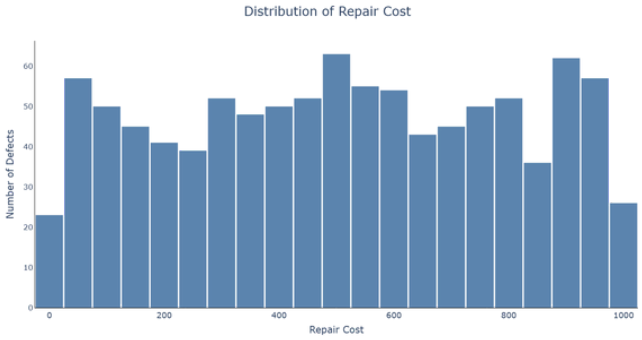
- Quality monitoring should consider structural, functional, and cosmetic defects rather than concentrating exclusively on one category.
- The relatively even distribution provides a basis for comparing defect characteristics and severity patterns across defect types.

Distribution of Repair Cost

33.33%

Traffic Type 16 Conversion





Key observations

- Repair costs extend across a broad range, approximately from 10 to 1,000.
- The distribution appears broadly uniform, without a pronounced concentration in a particular cost interval.

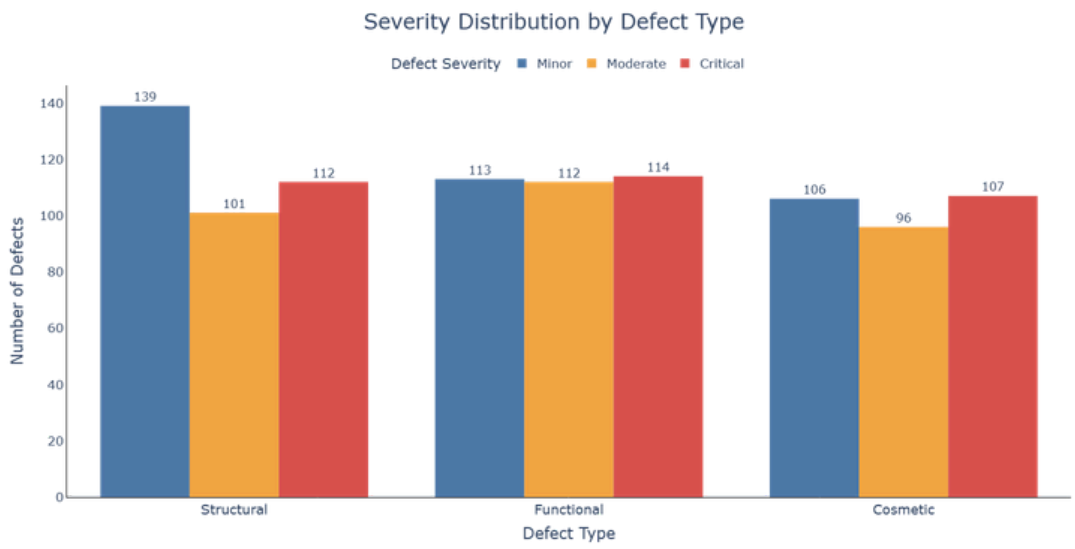
Business Insights

- The considerable variation in repair costs highlights the importance of understanding repair expenditure when planning manufacturing quality management activities.
- Additional operational information may be required to explain why some defects incur substantially higher repair costs than others.

Severity Distribution by Defect Type

~35

Product Pages Viewed (median)



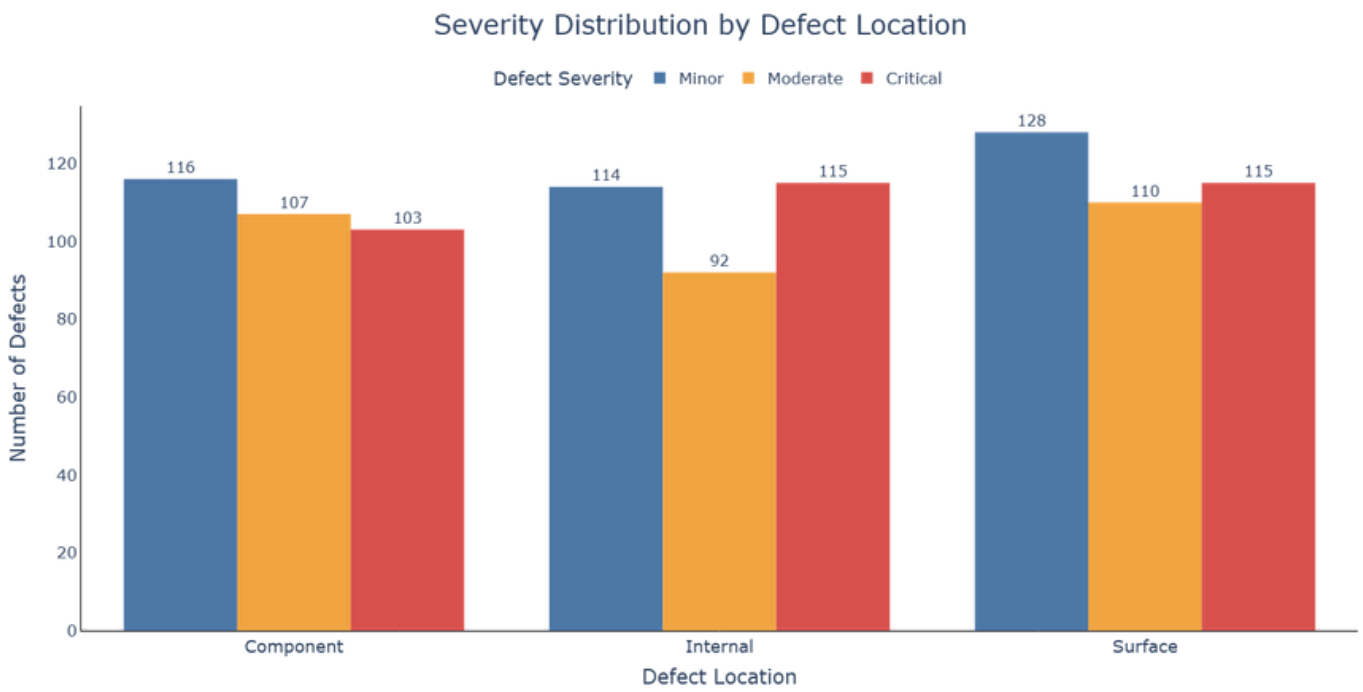
Key observations

- Structural defects have the highest Minor defect count, while Functional defects show nearly equal representation across all three severity categories.
- Critical defects occur across Structural, Functional, and Cosmetic categories without a pronounced concentration in one type.

Business Insights

- Defect type alone may provide limited information for distinguishing severity levels.
- Quality assurance procedures should evaluate severity within every defect category rather than assuming a particular defect type is inherently more critical.

Severity Distribution by Defect Location



Key observations

- Surface defects have the highest Minor defect count, while Internal defects show nearly equal Minor and Critical counts.
- Critical defects occur across Component, Internal, and Surface locations at relatively similar frequencies.

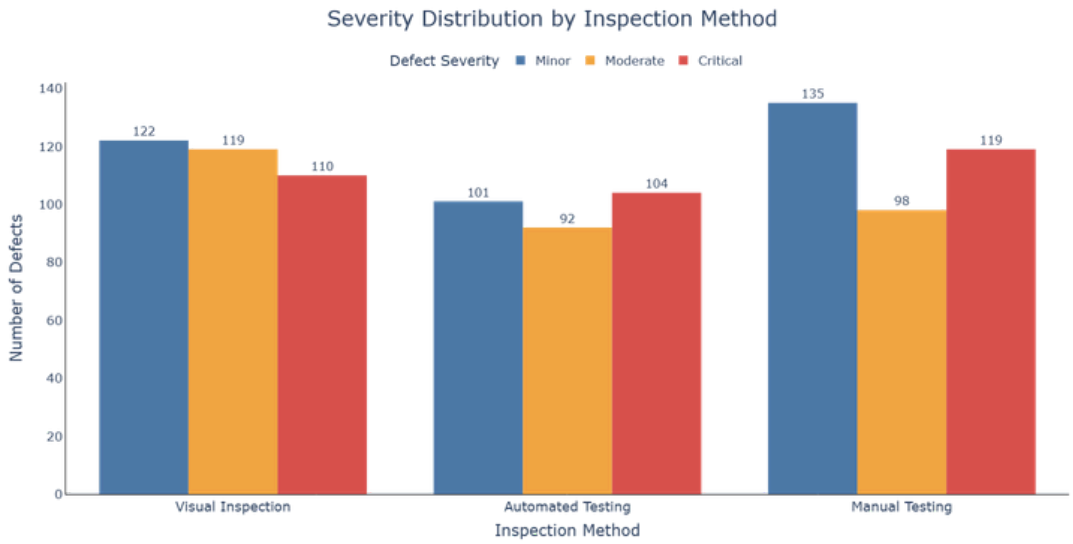
Business Insights

- Inspection coverage should include all defect locations because Critical defects are not restricted to a single area.
- Defect location may provide useful descriptive information but does not show strong visual separation between severity classes.

Severity Distribution by Inspection Method

52.03%

Typical CPU Utilization (Median)



Key observations

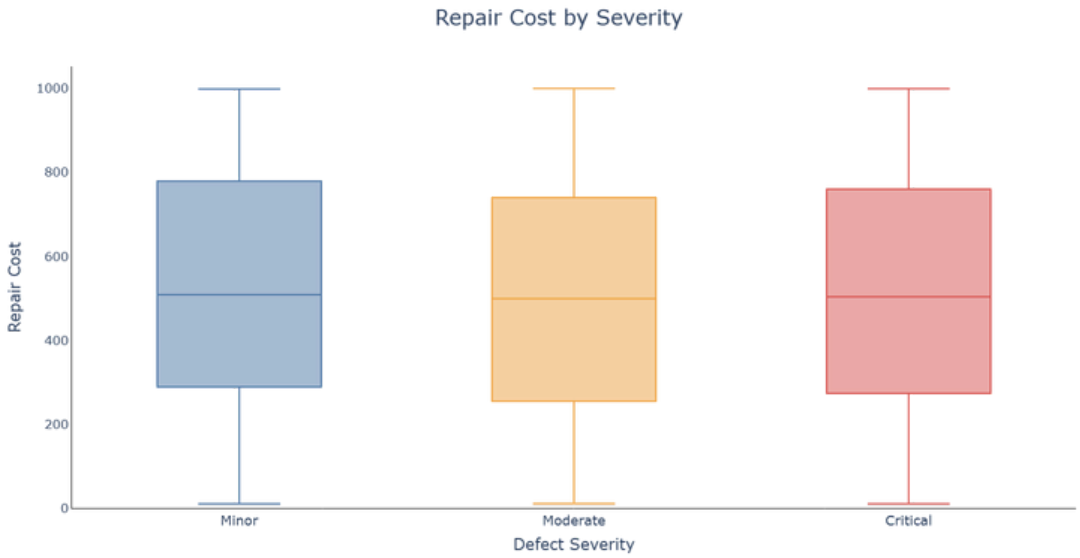
- Manual Testing records the highest Minor defect count, while Visual Inspection shows relatively similar counts across severity categories.
- Automated Testing has slightly more Critical than Minor defects, although differences between severity counts remain modest.

Business Insights

- All three inspection methods are associated with the identification of Critical, Minor, and Moderate defects.
- Differences in recorded counts should not be interpreted as evidence of inspection effectiveness without accounting for inspection volumes and detection conditions.

Repair Cost by Severity

0.0029
Median Bounce Rate



Key observations

- Median repair costs are approximately 500 across Minor, Critical, and Moderate defects.
- All three severity categories exhibit substantial cost variation and strongly overlapping distributions.

Business Insights

- Repair cost does not show a clear increasing pattern with defect severity in this dataset.
- Severity-based repair budgeting may require additional operational information beyond the recorded severity classification.

PageValues has a strong positive relationship with purchase conversion, while BounceRates ExitRates show negative relationships.



Key observations

- Repair costs vary widely within every combination of defect type and severity, with substantial overlap between groups.
- Median costs differ somewhat across combinations, but no consistent pattern indicates that Critical defects always incur higher repair costs.

Business Insights

- Neither defect type nor severity appears sufficient on its own to explain repair cost differences.
- More detailed information about repair procedures, defect complexity, and resource requirements would help investigate cost drivers.

Data Preparation & Feature Engineering



DATA PREPARATION

Following the exploratory analysis, the dataset was prepared for machine learning.

The preparation process included:

- Selecting severity as the target variable.
- Separating predictor variables from the target.
- Identifying numerical, categorical, and temporal variables.
- Assessing missing values and data quality.
- Excluding identifier variables (defect_id and product_id).
- Removing repair_cost to prevent potential target leakage.
- Applying an 80:20 stratified train-test split.
- Creating temporal features from defect_date.
- Preparing categorical variables for one-hot encoding.
- Standardizing numerical features.
- Building a reusable preprocessing pipeline.
- Validating and saving processed datasets for modeling.



3.1 FEATURE SELECTION

The target variable was separated from the predictive features, and variables unsuitable for the intended classification scenario were excluded.

The final selected predictors were:

Feature	Type
defect_type	Categorical
defect_location	Categorical
inspection_method	Categorical
defect_date	Temporal

The excluded variables were:

Feature	Reason for Exclusion
defect_id	Unique record identifier
product_id	High-cardinality identifier
repair_cost	Potential post-assessment target leakage

The target variable was separated from the predictive features, and variables unsuitable for the intended classification scenario were excluded.

3.2 TRAIN-TEST SPLIT

The dataset was divided into training and testing subsets using an 80:20 stratified split.

Dataset	Observations	Percentage
Training	800	80%
Testing	200	20%

Stratification preserved the approximate distribution of the three severity classes

A fixed random state of 42 was used to support reproducibility.

The held-out test dataset was reserved for final model evaluation and was not used during hyperparameter optimization.

The training and testing datasets were saved for reuse across subsequent notebooks, allowing model development, evaluation, and explainability to operate on consistent data.

3.3 TEMPORAL FEATURE ENGINEERING

The original defect_date variable was transformed into four temporal features:

Engineered Feature	Description
Month	Calendar month of the defect record
Day	Day of the month
Day_of_Week	Day of the week
Is_Weekend	Binary weekend indicator

These variables were introduced to investigate whether calendar-related information contributed to defect severity classification.

For example, temporal characteristics could potentially reflect changes in production schedules or inspection conditions. However, the dataset does not provide sufficient operational context to confirm such relationships.

After feature extraction, the original date column was removed from the modeling dataset.

The resulting feature set contained seven original and engineered predictors.



3.4 CATEGORICAL ENCODING

Three categorical variables were prepared using One-Hot Encoding:

- *defect_type*
- *defect_location*
- *inspection_method*

The encoding process transformed each categorical value into a separate binary indicator.

The three variables produced nine encoded features.

One-Hot Encoding was selected because it allows classification models to use categorical information without introducing artificial numerical ordering between categories.

The encoder was configured to handle previously unseen categories during transformation.

3.5 NUMERICAL SCALING

The numerical temporal variables were standardized using StandardScaler:

- *Month*
- *Day*
- *Day_of_Week*

Standardization transforms numerical variables using statistics learned from the training dataset.

This is particularly relevant for distance-based and margin-based classifiers, including K-Nearest Neighbors, Logistic Regression, and Support Vector Machines.

The binary Is_Weekend feature was retained without scaling.



3.6 PREPROCESSING PIPELINE

A Scikit-learn *ColumnTransformer* was used to combine categorical encoding, numerical scaling, and binary passthrough into a reusable preprocessing workflow.

The resulting feature representation contained:

Feature Group	Number of Features
One-hot encoded categorical features	9
Standardized numerical features	3
Binary features	1
Total processed features	13

The transformed dataset dimensions were:

Dataset	Shape
Training	800 × 13
Testing	200 × 13

The preprocessing transformer was fitted exclusively on training data before being applied to the held-out test set.

This prevented information from the test data from influencing the learned preprocessing parameters.



3.7 DATA VALIDATION

Following preprocessing, the transformed datasets were checked for missing and infinite values.

The validation confirmed that the processed training and testing datasets contained no missing or infinite values.

This established a consistent feature representation for the subsequent model development stage.

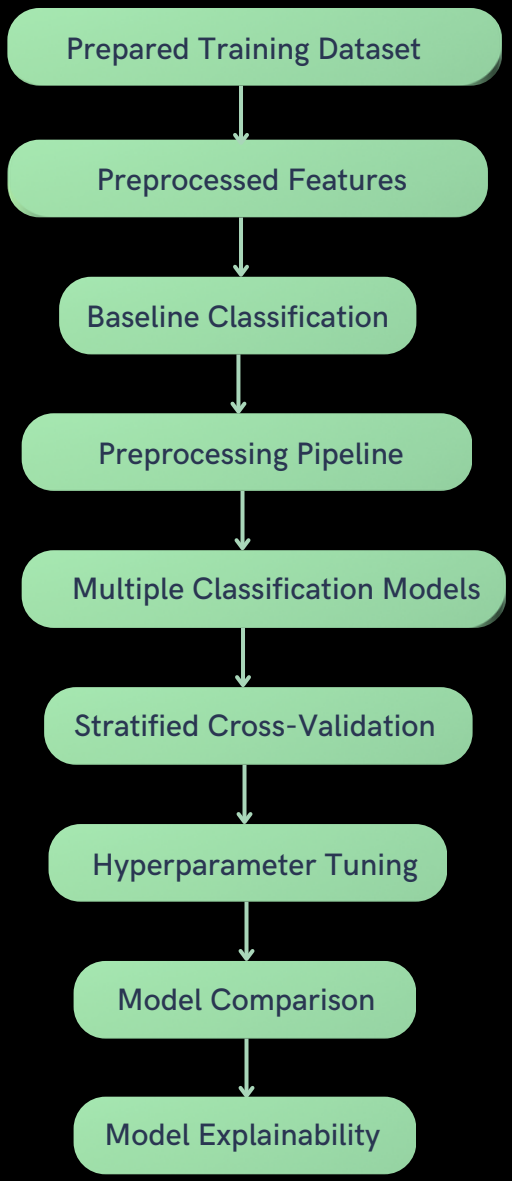
Model Development & Experimentation



4.1 MODELING APPROACH

Manufacturing defect severity classification was formulated as a supervised ***multiclass classification*** problem.

The modeling workflow followed:



The modeling objective was to estimate the severity category associated with each defect record using the selected manufacturing and temporal features.

Rather than relying on a single algorithm, the project evaluated multiple model families to investigate different approaches to learning relationships within the dataset.

These included linear classifiers, distance-based methods, tree-based algorithms, boosting methods, and ensemble classifiers.

A total of ***19 model*** configurations were evaluated.

4.2 BASELINE CLASSIFICATION

A Dummy Classifier was used to establish a simple reference point for model performance.

The baseline predicted the majority class without learning relationships between the predictive variables and defect severity.

Its cross-validation results were:

Metric	Score
Accuracy	0.3575
Macro F1	0.1756

The baseline provided an important comparison for determining whether more sophisticated models learned useful predictive patterns.

A classifier achieving only a marginal improvement over a simple baseline may have limited practical value, particularly when severe classification errors have operational consequences.



4.3 CLASSIFICATION ALGORITHMS

The project evaluated the following model families:

Model	Modeling Approach
Logistic Regression	Linear probabilistic classification
Decision Tree	Rule-based decision partitioning
Random Forest	Bagging ensemble of decision trees
Extra Trees	Randomized tree-based ensemble
Support Vector Machine	Margin-based classification
K-Nearest Neighbors	Distance-based classification
Gaussian Naive Bayes	Probabilistic classification
Gradient Boosting	Sequential boosting of weak learners
XGBoost	Regularized gradient-boosted trees

These algorithms were selected to compare different learning assumptions and investigate whether nonlinear relationships, local neighborhood structures, or ensemble methods improved classification performance.



4.4 CROSS-VALIDATION STRATEGY

Model development used 5-fold Stratified Cross-Validation.

Stratification preserved the approximate severity-class proportions within each fold.

The following evaluation metrics were recorded:

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1

Macro F1 was selected as the primary model comparison metric because it evaluates all three severity classes equally.

This is important in a manufacturing quality assurance context, where a model should not achieve favorable overall performance simply by predicting the most common category.

Critical defect recall was also considered because missed Critical defects represent an important operational risk.



4.5 HYPERPARAMETER OPTIMIZATION

Hyperparameter tuning was performed for five selected classifiers:

- Logistic Regression
- Random Forest
- Support Vector Machine
- K-Nearest Neighbors
- XGBoost

The objective was to identify configurations that improved cross-validation performance without introducing unnecessary computational complexity.



4.6 ENSEMBLE LEARNING

Three voting-based ensembles and one stacking ensemble were evaluated.

Hard Voting Ensemble

The Hard Voting Ensemble combined predictions from Tuned SVM, Tuned Logistic Regression, and Tuned Random Forest through majority voting.

Soft Voting Ensemble

The Soft Voting Ensemble combined predicted class probabilities from Tuned Logistic Regression, Tuned Random Forest, and Tuned XGBoost.

Weighted Soft Voting Ensemble

The Weighted Soft Voting Ensemble used the same probability-based approach while assigning different contributions to its constituent models.

Stacking Ensemble

The Stacking Ensemble combined predictions from multiple base classifiers through a meta-classifier.

These methods were included to investigate whether combining models with different learning characteristics could improve multiclass classification performance.



4.7 CROSS-VALIDATION MODEL COMPARISON

The complete cross-validation comparison is presented below.

Model	CV Accuracy	CV Macro F1
Tuned Support Vector Machine	0.3750	0.3710
Hard Voting Ensemble	0.3688	0.3646
Tuned Logistic Regression	0.3638	0.3612
Logistic Regression	0.3613	0.3582
Tuned Random Forest	0.3700	0.3567
Tuned K-Nearest Neighbors	0.3688	0.3564
Soft Voting Ensemble	0.3513	0.3442
Weighted Soft Voting Ensemble	0.3550	0.3440
Tuned XGBoost	0.3413	0.3350
Gaussian Naive Bayes	0.3313	0.3294
XGBoost	0.3250	0.3209
Decision Tree	0.3200	0.3187
Gradient Boosting	0.3213	0.3175
Support Vector Machine	0.3275	0.3153



Model	CV Accuracy	CV Macro F1
K-Nearest Neighbors	0.3150	0.3028
Random Forest	0.2963	0.2934
Stacking Ensemble	0.3413	0.2924
Extra Trees	0.2863	0.2841
Dummy Classifier	0.3575	0.1756

Interpretation

The Tuned Support Vector Machine achieved the highest cross-validation Macro F1-score of 0.3710, followed by the Hard Voting Ensemble at 0.3646.

Although several models improved upon the Dummy Classifier's Macro F1-score, their overall classification accuracy remained close to the class proportions observed in the dataset.

Ensemble learning did not consistently outperform individual classifiers.

These results suggest that the available features contain limited predictive information for distinguishing the three severity categories.



4.8 EXPERIMENT TRACKING

MLflow was used to organize machine learning experiments and record model development results.

The experiment tracking workflow supported:

- Recording model configurations
- Tracking cross-validation metrics
- Comparing modeling experiments
- Organizing model development results

Trained model artifacts were saved using Joblib for reuse in the evaluation and explainability notebooks.

This approach improved reproducibility and reduced the need to retrain models during subsequent analysis.



Model Evaluation & Comparison

5.1 EVALUATION STRATEGY

After model development, all trained configurations were evaluated on the held-out test dataset containing 200 observations.

The test dataset was not used for hyperparameter optimization.

This separation was important for assessing whether the models could generalize beyond the observations used during training and cross-validation.

The evaluation focused on overall classification performance, class-specific behavior, Critical defect detection, and differences between cross-validation and test results.



5.2 EVALUATION METRICS

The classification models were evaluated using:

Accuracy

Accuracy measures the proportion of correctly classified observations. Although useful for overall performance assessment, accuracy does not distinguish between errors involving Critical, Minor, and Moderate defects.

Macro Precision

Macro Precision calculates the average precision across all severity classes. It indicates how reliably the model's predicted severity labels correspond to actual observations.

Macro Recall

Macro Recall calculates the average recall across all three severity categories. It evaluates the model's ability to identify observations belonging to each class.

Macro F1-Score

Macro F1 combines precision and recall into a balanced measure and averages the class-level F1-scores equally. It was selected as the primary performance metric because each severity class is important to the classification problem.

Critical Recall

Critical Recall measures the proportion of actual Critical defects correctly identified by the model. This metric received additional attention because missed Critical defects may have greater operational consequences than some other classification errors.

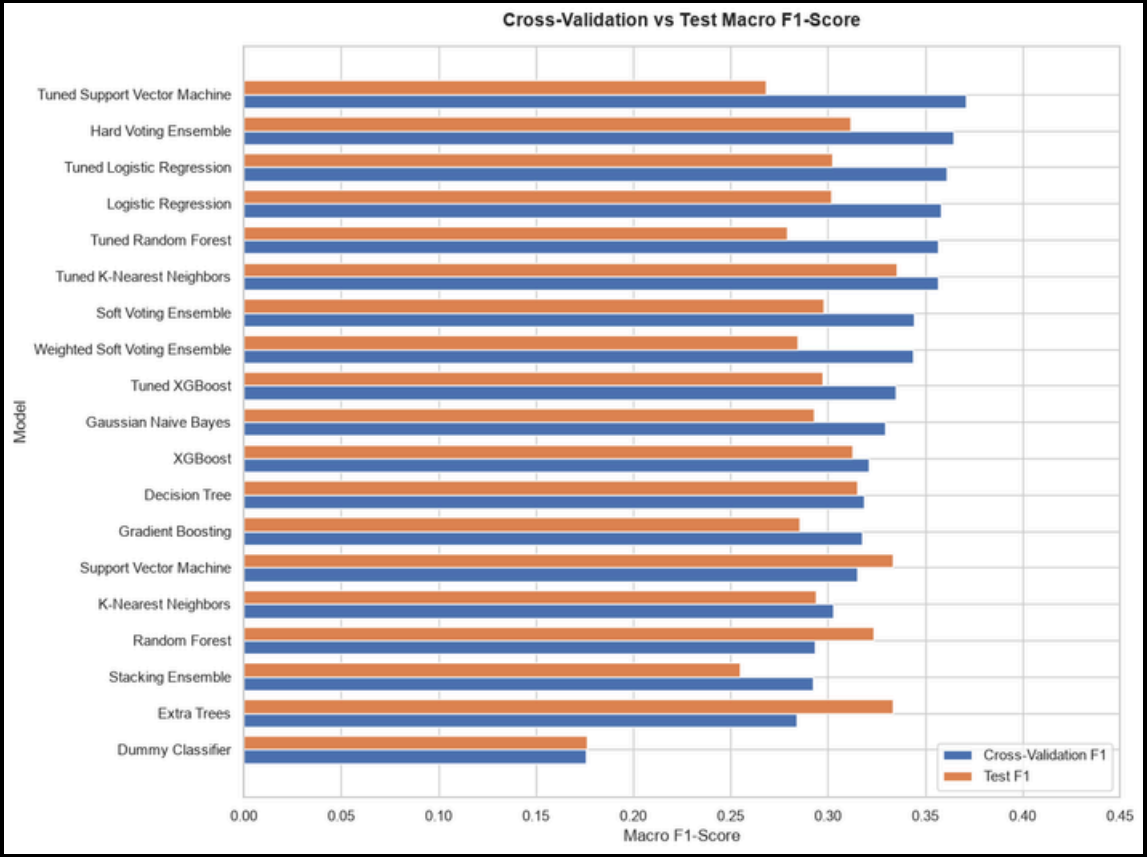
5.3 HELD-OUT TEST PERFORMANCE

The test results for all 19 configurations are shown below.

Model	Test Accuracy	Test Macro F1	Critical Recall
Tuned K-Nearest Neighbors	0.3500	0.3354	0.3939
Support Vector Machine	0.3550	0.3335	0.3636
Extra Trees	0.3550	0.3333	0.3333
Random Forest	0.3300	0.3236	0.3636
Decision Tree	0.3150	0.3150	0.3333
XGBoost	0.3200	0.3126	0.3030
Hard Voting Ensemble	0.3300	0.3118	0.4242
Tuned Logistic Regression	0.3100	0.3020	0.3939
Logistic Regression	0.3100	0.3019	0.3939
Soft Voting Ensemble	0.3100	0.2978	0.3333
Tuned XGBoost	0.3050	0.2973	0.2879
K-Nearest Neighbors	0.3100	0.2938	0.5000
Gaussian Naive Bayes	0.2950	0.2927	0.3636
Gradient Boosting	0.2900	0.2853	0.3182



5.4 CROSS-VALIDATION VS TEST PERFORMANCE



A comparison between cross-validation and test performance was performed to investigate model generalization.

Model	Test Accuracy	Test Macro F1	Critical Recall
Tuned SVM	0.3710	0.2679	0.1031
Tuned KNN	0.3564	0.3354	0.0210
Hard Voting Ensemble	0.3646	0.3118	0.0528
Tuned Logistic Regression	0.3612	0.3020	0.0592

Interpretation

The Tuned SVM achieved the highest cross-validation Macro F1-score but experienced a substantial reduction in performance on the held-out test dataset.

Its Macro F1-score declined from 0.3710 to 0.2679, representing a difference of approximately 0.1031.

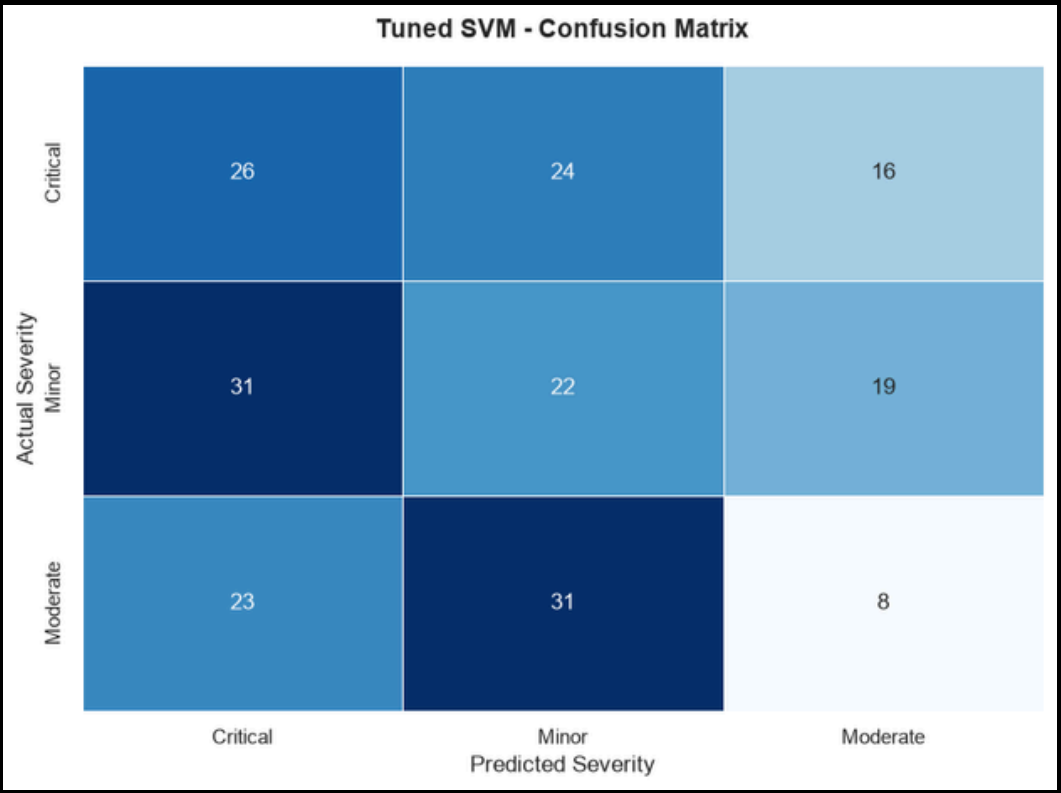
Tuned KNN showed a smaller difference between cross-validation and test performance.

These findings indicate that the model selected through cross-validation did not maintain its relative performance on unseen observations.

Given the limited dataset size, some variation between cross-validation and test results is expected. Nevertheless, the weak absolute performance remains an important limitation.



5.5 CONFUSION MATRIX ANALYSIS



The Tuned SVM confusion matrix provides a detailed view of classification errors across the three severity categories.

Actual / Predicted	Test Accuracy	Test Macro F1	Critical Recall
Critical	26	24	16
Minor	31	22	19
Moderate	23	31	8

The model correctly identified 26 of the 66 Critical defects.

- 66 Critical defects
- 72 Minor defects
- 62 Moderate defects

Interpretation

The model correctly identified 26 of the 66 Critical defects.

However, 40 actual Critical defects were incorrectly classified as Minor or Moderate.

The model also showed considerable confusion between Minor and Moderate defects.

Only eight Moderate defects were correctly identified, indicating particularly weak recognition of this severity category.

Overall, the confusion matrix demonstrates that the model struggled to establish reliable decision boundaries between the three severity classes.

